CIS 3300 · CLASSES AND OBJECTS REVIEW, PART 1 OF 3

Static Members and Nested Classes

What belongs to the class, what belongs to an object, and classes inside classes

Before we start inheritance, we are going to tidy up a few corners of classes and objects. This is the first of three short sets of notes. It covers what the word `static` really means, how to set up static data that takes more than one line, and how Java lets you put one class inside another. The examples build a small course roster, and the next two sets of notes keep using it. For reading, see Core Java, Volume I, Chapter 4 (static fields and methods) and Chapter 6 (inner classes).

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. Instance Members and Class Members

Most fields you have written so far are *instance fields*. Every object gets its own copy, so two `Student` objects can have different names and different scores. A *static field* is different: it belongs to the class itself. There is exactly one copy, every object shares it, and it exists even before any object has been created.

Here is the `Student` class we will use in this set of notes. It hands out IDs automatically.

```
public class Student {
    // Shared by every Student: one copy for the whole class.
    private static int nextId = 1;
    public static final String ID_PREFIX = "S";

    // One copy per object.
    private final String id;
    private final String name;
    private double percent;

    public Student(String name, double percent) {
        this.id = ID_PREFIX + nextId;
        nextId++;
        this.name = name;
        this.percent = percent;
    }
```

```
    public static int getStudentsCreated() {
        return nextId - 1;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public double getPercent() { return percent; }
    public void setPercent(double percent) { this.percent = percent; }

    @Override
    public String toString() {
        return id + " " + name + " (" + percent + ")";
    }
}
```

Three static members are worth a closer look.

- `nextId` is a shared counter. Each constructor call reads it and then increments it, so each new student gets the next number. If `nextId` were an instance field, every new object would start its own counter at 1, and every student would be `S1`.
- `ID_PREFIX` is a class constant: its value never changes, and it is the same for every `Student`. Declaring it `static final` gives the class one shared copy that code can use as `Student.ID_PREFIX` without creating an object. Compare it with `id` and `name`, which are also `final` but are instance fields, because each student has its own unchanging value. You already know some class constants: `Math.PI` and `Integer.MAX_VALUE`.
- `getStudentsCreated()` is a static method. You call it on the class itself, and no object is needed: `Student.getStudentsCreated()`.

```
Student ada = new Student("Ada", 91.5);
Student grace = new Student("Grace", 84.0);
System.out.println(ada.getId());                    // S1
System.out.println(grace.getId());                  // S2
System.out.println(Student.getStudentsCreated());   // 2
```

Figure 1 shows what memory looks like at this point.

What a static method cannot do

A static method runs without an object, so it has no `this`. That means it cannot touch instance fields or call instance methods directly, because there is no particular object whose fields it would be using. Try it and the compiler tells you exactly that:

```
public static int getStudentsCreated() {
    return name.length();   // which student's name?
}
```

```
error: non-static variable name cannot be referenced from a static context
```

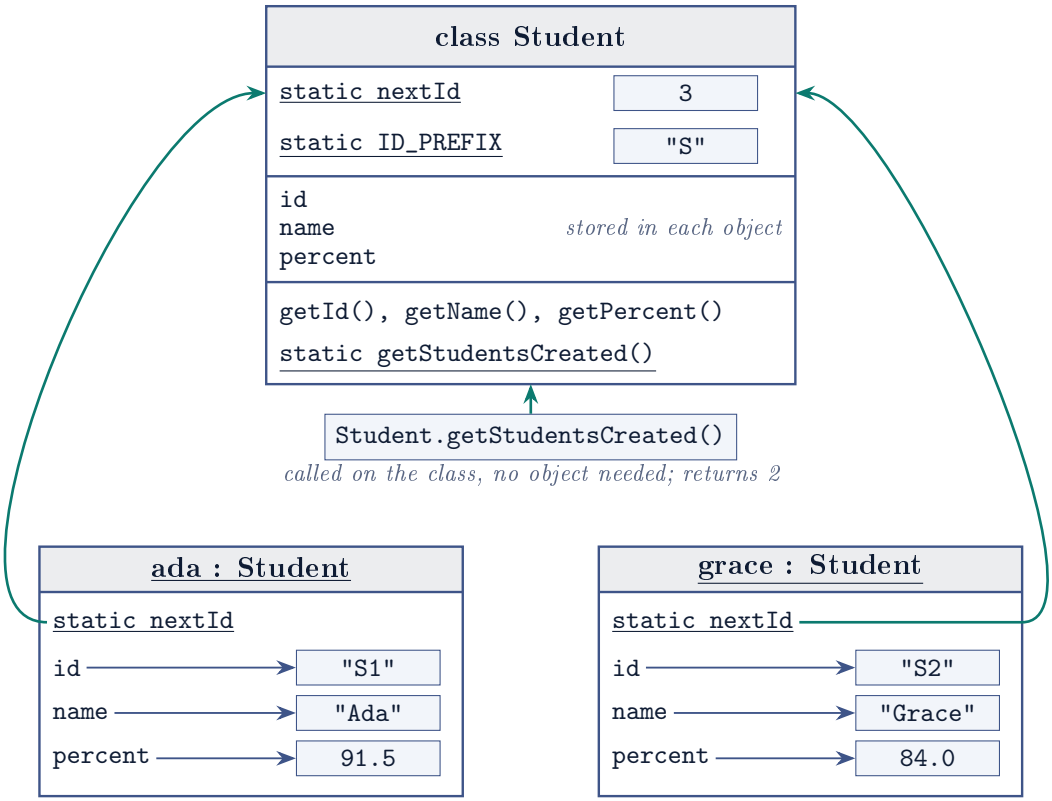


Figure 1: One copy of each static member, shared by every object. Underlined names are static, following the UML convention. Before any `Student` existed, `nextId` started at 1. Creating `ada` and `grace` incremented it twice, so it now holds 3, the next ID to hand out. Each object stores its own `id`, `name`, and `percent`, but its `nextId` is only a path to the single copy in the class. The static method is called on the class itself.

Table 1: What instance methods and static methods can reach directly.

Code running in	Static fields and methods	Instance fields and methods
An instance method	Yes	Yes, through `this`
A static method	Yes	Only through an object reference you have

This is also why `main` is static. The JVM calls `main` before any of your objects exist, so it needs a method it can call on the class itself. Other static methods you have already used include `Math.sqrt`, `Integer.parseInt`, and `Arrays.sort`.

Access static members in a static way

Java lets you reach a static member through an object reference, as in `ada.getStudentsCreated()` or `ada.ID_PREFIX`. It compiles and runs, but it is misleading, because it looks like the answer depends on `ada`. It does not. Figure 1 shows why: `ada`'s `nextId` entry is only an arrow to the one copy in the class, so going through `ada` just leads back to the class.

Eclipse flags this with a warning, marked by a yellow triangle in the editor and listed in the Problems view:

```
The static method getStudentsCreated() from the type Student should be accessed in a
    static way
The static field Student.ID_PREFIX should be accessed in a static way
```

The standard `javac` compiler says nothing by default. If you ask for these warnings with the `-Xlint:static` option, it reports the same problem in its own words:

```
$ javac -Xlint:static StaticWayDemo.java
StaticWayDemo.java:12: warning: [static] static method should be qualified by type name
    , Student, instead of by an expression
```

Here is proof that the object really is ignored. Calling an instance method through a `null` reference throws a `NullPointerException`, but this call works:

```
Student nobody = null;
System.out.println(nobody.getStudentsCreated());   // prints 2; no exception
```

The compiler uses only the variable's declared type, `Student`, to decide which method to call. It never looks at the object, so it never notices that there is no object at all.

The fix is to name the class instead of an object.

Table 2: Accessing static members in a static way.

Where the code is	Non-static way (warning)	Static way (no warning)
Outside `Student`	`ada.getStudentsCreated()`	`Student.getStudentsCreated()`
Outside `Student`	`ada.ID_PREFIX`	`Student.ID_PREFIX`
Inside `Student`	`this.nextId`	`nextId` or `Student.nextId`

Inside the `Student` class itself, the bare names `nextId` and `ID_PREFIX` are fine and produce no warning. Writing `this.nextId` does produce one, for the same reason: `this` is an object reference. In Eclipse, click the warning and press Ctrl+1 (Cmd+1 on a Mac). The quick fix rewrites the access to use the class name for you.

Course expectation. Programs you submit in this class should not produce this warning. Before you submit, check the Problems view in Eclipse for any "should be accessed in a static way" warnings and fix each one by using the class name.

When to use static, and when not to

Good uses of static members:

- Constants, such as `ID_PREFIX`.
- Data that truly belongs to the whole class, such as the `nextId` counter.
- Utility methods whose result depends only on their parameters, such as `Math.max`.
- Factory methods that create objects for you. We will write one in the next set of notes.

The most common misuse comes from `main`. You call an instance method from `main`, the compiler says "non-static ... cannot be referenced from a static context," and the quick fix seems to be adding `static` to everything. That makes the error go away, but now every object shares one copy of the data, which is almost never what you meant. The real fix is to create an object in `main` and call the method on it.

A mutable static field is also a global variable in disguise: any code anywhere can change it. Use static for things that are shared by design, not to avoid passing an object around.

2. Static Initialization Blocks

A simple static field can be set in one line, like `nextId = 1`. Sometimes setting up a static field takes real work: reading a file, filling a collection, or handling an error. A *static initialization block* is a block of code marked `static` that runs once to do that setup.

In this example, the maximum roster size comes from a small text file if one exists, and falls back to a default otherwise.

```
import java.io.IOException;
import java.nio.file.Path;
import java.util.NoSuchElementException;
import java.util.Scanner;

public class Config {
    public static final int MAX_ROSTER_SIZE;

    static {
        System.out.println("Config: static block running");
        int size = 30;   // default if the file is missing or unreadable
        try (Scanner in = new Scanner(Path.of("config.txt"))) {
            size = in.nextInt();
        } catch (IOException | NoSuchElementException e) {
            System.out.println("Config: using the default roster size");
        }
        MAX_ROSTER_SIZE = size;
    }
}
```

Points to notice:

- **It runs once.** The block runs when the class is first initialized, which happens the first time the program actually uses the class, not when the program starts. In the demo at the end of these notes, `"main starts"` prints before the block runs, and the block prints only once even though `Config.MAX_ROSTER_SIZE` is used several times.
- **A static final can be assigned here.** `MAX_ROSTER_SIZE` is declared without a value, and the block assigns it exactly once. The compiler checks that every path through the block assigns it.
- **Order matters.** Static field initializers and static blocks run from top to bottom, in the order they appear in the file.
- **Handle errors inside the block.** A checked exception such as `IOException` must be caught inside the block. If an unchecked exception escapes, the class fails to load properly. Java reports

it as an `ExceptionInInitializerError`, and the class cannot be used for the rest of the run. That is why the example catches the problem and falls back to a default.

3. Classes Inside Classes

You may have noticed that you cannot write `private class` or `static class` at the top of a file. Try it and the compiler says:

```
error: modifier private not allowed here
```

Those modifiers apply to *members* of a class, and a class can only be a member if it is declared inside another class. A class declared inside another class is a *nested class*. Java has four kinds.

Table 3: The four kinds of nested classes.

Kind	Where it is declared	Tied to an outer object?	Typical use
Static nested class	Inside a class, marked `static`	No	A helper type that belongs with the outer class
Inner class	Inside a class, not `static`	Yes	A helper that needs the outer object's data
Local class	Inside a method	Yes, when the method is not static	A helper used by only one method
Anonymous class	Inside an expression, with no name	Yes, when created in an instance method	A one-time implementation of an interface

Like any member, a nested class can be `public`, `private`, `protected`, or have no modifier. A nested class can also read the `private` members of the class around it, and the outer class can read the nested class's `private` members. They are all part of the same class body.

4. Static Nested Classes

A `Roster` needs to report summary statistics: how many students, the lowest and highest percent, and the average. That summary is its own small type, but it only makes sense alongside a roster, so we nest it inside `Roster`.

```
import java.util.ArrayList;

public class Roster {
    private final String courseCode;
    private final ArrayList<Student> students = new ArrayList<>();

    public Roster(String courseCode) {
        this.courseCode = courseCode;
    }
```

```
    public void add(Student s) {
        if (students.size() >= Config.MAX_ROSTER_SIZE) {
            throw new IllegalStateException(courseCode + " is full");
        }
        students.add(s);
    }

    // ----- Static nested class: no Roster object required -----
    public static class Stats {
        private final int count;
        private final double low;
        private final double high;
        private final double average;

        public Stats(int count, double low, double high, double average) {
            this.count = count;
            this.low = low;
            this.high = high;
            this.average = average;
        }

        public int getCount() { return count; }
        public double getLow() { return low; }
        public double getHigh() { return high; }
        public double getAverage() { return average; }

        @Override
        public String toString() {
            return count + " students, low " + low + ", high " + high
                    + ", average " + average;
        }
    }

    public Stats computeStats() {
        if (students.isEmpty()) {
            return new Stats(0, 0.0, 0.0, 0.0);
        }
        double low = students.get(0).getPercent();
        double high = low;
        double sum = 0.0;
        for (Student s : students) {
            low = Math.min(low, s.getPercent());
            high = Math.max(high, s.getPercent());
            sum += s.getPercent();
        }
        return new Stats(students.size(), low, high, sum / students.size());
    }

    // The inner class Report is in the next section.
}
```

A static nested class behaves like an ordinary top-level class that happens to live inside `Roster`'s name. Outside code refers to it as `Roster.Stats`, and it can create one without having a `Roster`

at all:

```
Roster.Stats stats = roster.computeStats();             // from a Roster
Roster.Stats manual = new Roster.Stats(2, 70.0, 90.0, 80.0);   // no Roster needed
```

Because a `Stats` object is not tied to any particular roster, code inside `Stats` cannot use `Roster`'s instance fields. Try to read `students` from a method in `Stats` and you get the same error you saw for static methods:

```
error: non-static variable students cannot be referenced from a static context
```

The word `static` means the same thing here as it does on a field: the nested class belongs to `Roster` the class, not to any one `Roster` object.

5. Inner Classes

An *inner class* is a nested class without `static`. Every inner class object is created by, and permanently linked to, one object of the outer class. Through that hidden link it can read the outer object's fields directly, including `private` ones.

Suppose a roster should be able to print a numbered report of its students. Here is the rest of `Roster`:

```
    public void printReport() {
        Report report = new Report();   // linked to this Roster object
        report.print();
    }

    // ----- Private inner class: tied to one Roster object -----
    private class Report {
        private int lineNumber = 1;

        void print() {
            System.out.println("Report for " + courseCode);   // outer Roster's field
            for (Student s : students) {                      // outer Roster's list
                System.out.println(lineNumber + ". " + s);
                lineNumber++;
            }
        }
    }
```

Things to notice:

- `Report` uses `courseCode` and `students` as if they were its own fields. They are really the fields of the `Roster` that created the report. The long way to write them is `Roster.this.courseCode` and `Roster.this.students`, which you need only when a name in the inner class hides a name in the outer class.
- Inside `printReport()`, `new Report()` quietly links the new report to `this` roster. Two rosters produce two reports, and each report sees only its own roster's students. The demo in Section 8 shows this.

- `Report` also has a field of its own, `lineNumber`, so every report starts numbering at 1.
- The class is `private`. Code outside `Roster` cannot see it or create one. Hiding the class is encapsulation at the class level, the same idea as a private field.

Is an inner class really needed here? Honestly, no. In a program this small, an ordinary private method in `Roster` would do the same job. Inner classes earn their keep in larger designs, such as the iterator objects that make a for-each loop over an `ArrayList` work, and the event listeners you will see in GUI programs. A small example keeps the focus on the important part: the link between an inner object and its outer object.

You will rarely need it, but if an inner class were `public`, outside code would need an outer object to create one, using the unusual syntax `roster.new SomeInnerClass()`.

6. Choosing Between Static Nested and Inner

Start with a static nested class. Remove `static` only if the nested class truly needs to read the fields of a specific outer object, as `Report` does.

The reason is that every inner class object carries a hidden reference to its outer object. That costs a little memory, and more importantly it keeps the outer object alive as long as the inner object exists, even if nothing else is using the outer object. A `Stats` object handed to some other part of the program should not drag the whole roster along with it, so `Stats` is static.

7. Local and Anonymous Classes, Briefly

A *local class* is declared inside a method and can be used only in that method. An *anonymous class* goes one step further: it is declared and created in a single expression and has no name at all. You will see anonymous classes in older Java code, so it helps to recognize one.

Here is a situation where one fits. First, a small interface describes a test that a student either passes or fails:

```
public interface StudentFilter {
    boolean accept(Student s);
}
```

Next, `Roster` gets a method that prints only the students who pass a given test:

```
    public void printIf(StudentFilter filter) {
        for (Student s : students) {
            if (filter.accept(s)) {
                System.out.println(s);
            }
        }
    }
```

To print the honor roll, `printIf` needs an object that implements `StudentFilter`. You could write a named class in its own file, but for a test you use only once, an anonymous class declares the class and creates the object in a single expression:

```
cis.printIf(new StudentFilter() {      // "a new object of an unnamed class
    @Override                          //  that implements StudentFilter"
    public boolean accept(Student s) {
        return s.getPercent() >= 90.0;
    }
});   // note the closing ");": this whole thing is one method argument
```

Local and anonymous classes can also use the local variables of the method around them, as long as those variables are never changed after they are set (Java calls this *effectively final*). In Week 14, lambda expressions will replace most uses of anonymous classes with much shorter code. The whole anonymous class above becomes `s -> s.getPercent() >= 90.0`.

8. Putting It Together

```
public class RosterDemo {
    public static void main(String[] args) {
        System.out.println("main starts");
        Roster cis = new Roster("CIS 3300");
        cis.add(new Student("Ada", 91.5));
        cis.add(new Student("Grace", 84.0));
        cis.add(new Student("Alan", 72.5));

        Roster math = new Roster("MATH 2100");
        math.add(new Student("Edsger", 77.0));
        System.out.println("Students created: " + Student.getStudentsCreated());

        cis.printReport();
        math.printReport();

        Roster.Stats stats = cis.computeStats();
        System.out.println(stats);

        System.out.println("Honor roll for CIS 3300:");
        cis.printIf(new StudentFilter() {
            @Override
            public boolean accept(Student s) {
                return s.getPercent() >= 90.0;
            }
        });

        System.out.println("Max size: " + Config.MAX_ROSTER_SIZE);
    }
}
```

Output, with no `config.txt` file present:

```
main starts
Config: static block running
Config: using the default roster size
Students created: 4
Report for CIS 3300
```

```
1. S1 Ada (91.5)
2. S2 Grace (84.0)
3. S3 Alan (72.5)
Report for MATH 2100
1. S4 Edsger (77.0)
3 students, low 72.5, high 91.5, average 82.66666666666667
Honor roll for CIS 3300:
S1 Ada (91.5)
Max size: 30
```

Table 4: Summary of the ideas in these notes.

Idea	Key point
Static field	One copy shared by the whole class, such as a counter or a constant
Static method	Called on the class; has no `this`, so it cannot use instance fields directly
Static block	Runs once, when the class is first used; good for setup that takes more than one line
Static nested class	A helper type inside another class, not tied to an outer object
Inner class	A helper type linked to one outer object; can read that object's fields
Anonymous class	A one-time, unnamed implementation of an interface, such as `StudentFilter`; mostly replaced by lambdas (Week 14)

9. Try It

1. Add a static field to `Roster` that counts how many rosters have been created, plus a static method that returns the count. Where must the count be incremented, and why does it have to be there?
2. Remove the word `static` from `Stats` so it becomes an inner class, then compile `RosterDemo` with the line `new Roster.Stats(2, 70.0, 90.0, 80.0)` added. Read the error message and explain it in your own words.
3. Create a `config.txt` file containing the number `2` and run `RosterDemo` again. Explain the result, including the order of the printed lines.